

ML Problem Solving

Vector Norms, Bias/Variance Trade-Off, SciKitLearn, Regularization

Part 1: Vector Norms

Consider the following vectors $v_i = \{a, b, c, d\} \subset \mathbb{R}^2$:

$$a = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad b = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \quad c = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad d = \begin{bmatrix} -3 \\ -4 \end{bmatrix}$$

Task 1.1: Visual Representation & Metric Calculations

1. **Plot the vectors:** Sketch vectors a, b, c, d on a single Cartesian coordinate plane with origin $(0, 0)$.
2. **Compute the L_2 norm (Euclidean norm)** for each vector:

$$\|v\|_2 = \sqrt{\sum_{i=1}^k v_i^2}$$

- $\|a\|_2 =$

- $\|b\|_2 =$

- $\|c\|_2 =$

- $\|d\|_2 =$

3. **Compute the L_1 norm (Manhattan / Taxicab norm)** for each vector:

$$\|v\|_1 = \sum_{i=1}^k |v_i|$$

- $\|a\|_1 =$

- $\|b\|_1 =$

- $\|c\|_1 =$

- $\|d\|_1 =$

4. **Generalization to L_p and L_∞ norms:**

- Define the general L_p norm

- Compute the Chebyshev / L_∞ norm $\|v\|_\infty = \max_i |v_i|$ for a, b, c, d .

Task 1.2: Angular Relationships & Directional Projections

1. Compute the **Cosine Similarity** between pairs:

$$\text{cos_sim}(u, v) = \frac{u^\top v}{\|u\|_2 \|v\|_2} = \cos(\theta)$$

- $\text{cos_sim}(a, b) =$

- $\text{cos_sim}(c, d) =$

- $\text{cos_sim}(a, c) =$

2. What is the geometric interpretation of $\text{cos_sim}(b, d)$? Show the relation to orthogonal projection.

3. Compute the normalized unit vectors $\hat{v} = \frac{v}{\|v\|_2}$ and $\tilde{v} = \frac{v}{\|v\|_1}$ for each vector. Explain how scale invariance affects distance-based and gradient-based learning algorithms.

Task 1.3: Isometric Unit Balls and Constraint Regions

1. L_2 Unit Ball:

- Write out the parametric or explicit algebraic equation for $\{x \in \mathbb{R}^2 : \|x\|_2 = 1\}$.

- Choose 10 vectors along the boundary and write them down.

- Describe the topology and geometry of the feasible region $\{x \in \mathbb{R}^2 : \|x\|_2 \leq 1\}$.

2. L_1 Unit Ball:

- Write out the piecewise equations for $\{x \in \mathbb{R}^2 : \|x\|_1 = 1\}$ across all four quadrants.

- Choose 10 vectors along the boundary and write them down.

- Contrast the shape (polytope / cross-polytope vs. hypersphere) and identify the points of non-differentiability (singularities).

Part 2: Noise, Complexity, and the Bias-Variance Tradeoff

Task 2.1: Deterministic vs. Stochastic Noise

In the lecture slides, two experimental curves of Expected E_{out} vs. Regularization Parameter λ are analyzed: * **Left graph:** Shows curves parameterized by variance of random label noise $\sigma^2 \in \{0, 0.25, 0.5\}$. * **Right graph:** Shows curves parameterized by target function complexity degree $Q_f \in \{15, 30, 100\}$ fitted with a lower-order model.

1. Distinguish between **stochastic noise** and **deterministic noise**.
2. For $\sigma^2 = 0$ (zero random noise) and low model complexity, where is the optimal λ^* located on the curve? Why does $\lambda^* \rightarrow 0$?
3. As deterministic noise (Q_f) or stochastic noise (σ^2) increases, explain why the optimal regularizer strength λ^* shifts to the right ($\lambda^* > 0$).
4. What happens to the model when $\lambda < \lambda^*$ vs. $\lambda > \lambda^*$? Relate this explicitly to the **bias-variance tradeoff**.

Part 3: Scikit-Learn Implementation

Task 3.1: Regression Pipeline & Polynomial Overfitting

For tasks 3.1 to 3.2, consider a tabular dataset $(\mathbf{X}, \mathbf{y})$ where y represents housing values. Obtain the data from the California Housing dataset available from SKLearn:

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html

Create a training split with 70% of the data and a test split with the rest.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, KFold, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet, LogisticRegression
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, log_loss, roc_auc_score
```

Pipeline Construction Construct an end-to-end Scikit-Learn Pipeline executing three stages:

1. StandardScaler()
2. PolynomialFeatures(degree=2, include_bias=False)
3. LinearRegression()

Fit the pipeline on $X_{\text{train}}, y_{\text{train}}$.

Compute and report:

- Training R^2 score and Test R^2 score.

- Training RMSE and Test RMSE.

Diagnostic Analysis: * In the lecture slide, the Polynomial Linear Regression achieves Training $R^2 \approx 95.16\%$ and Test $R^2 \approx 64.63\%$.

- Is this pipeline suffering from high bias or high variance? Justify your diagnosis.

Task 3.2: Comparative Regularization Study (Ridge vs. Lasso vs. ElasticNet)

Replace the linear estimator in the pipeline with: 1. `Ridge(alpha=10.0)` 2. `Lasso(alpha=0.3)` 3. `ElasticNet(alpha=0.1, l1_ratio=0.5)`

Questions & Parameter Auditing:

1. Extract the learned coefficient vectors (`pipeline.named_steps['model'].coef_`) from both the Ridge and Lasso models.
 - Count the total number of features generated by `PolynomialFeatures`.
- Count the exact number of coefficients identically equal to zero ($|w_j| < 10^{-6}$) in Lasso vs. Ridge.
- Plot a bar chart comparing the coefficient magnitudes of the first 25 features for both models.
2. What is the hyperparameter naming correspondence between the lecture mathematical formulas and Scikit-Learn?
 - In math: $\min L(w) + \lambda R(w)$.
 - In `sklearn.linear_model.Ridge`: what parameter controls λ ?
- In `sklearn.linear_model.LogisticRegression`: what parameter controls λ , and what is its functional relationship to λ ?

Task 3.3: Systematic Hyperparameter Tuning with Cross-Validation

Do not evaluate hyperparameter choices on your hold-out test set! Implement 5-fold cross-validation using `GridSearchCV`. 1. Execute `GridSearchCV` with scoring metric `neg_mean_squared_error`.

2. Extract `cv_results_` and plot the validation curve: **Mean CV Test Score vs. $\log(\alpha)$** .
3. Identify the optimal α^* for both Ridge and Lasso.
4. Compare performance on the final untouched test set: Did the tuned regularized model outperform unregularized polynomial regression?

Task 3.4: Regularized Logistic Regression for Classification

Synthesize classification theory and practice. Load or generate a synthetic classification dataset with collinear features or high dimensions:

```
from sklearn.datasets import make_classification

X_cls, y_cls = make_classification(
    n_samples=250,
    n_features=50,
    n_informative=10,
    n_redundant=15,
    n_clusters_per_class=2,
    weights=[0.6, 0.4],
    random_state=101
)
X_c_train, X_c_test, y_c_train, y_c_test = train_test_split(
    X_cls, y_cls, test_size=0.3, random_state=101, stratify=y_cls
)
```

1. **Build a Pipeline:** StandardScaler → LogisticRegression.
2. **Compare Solvers and Penalties:**
 - Fit with `penalty='l2', C=1.0, solver='lbfgs'`.
 - Fit with `penalty='l1', C=1.0, solver='saga'`.
 - Fit with `penalty=None` (or `C=1e5`, unregularized).
3. **Performance Metrics:** Compute the following metrics on test data for each model:
 - Binary Cross-Entropy (`log_loss`)
 - Accuracy (`accuracy_score`)
 - Area Under the ROC Curve (`roc_auc_score`)
4. **Inspect Weights:** Demonstrate that the L_1 penalty successfully zeroes out the coefficients corresponding to the non-informative features.

Part 4: Regularization

Group Discussion Questions

Record your group's synthesized answers to the following three foundational questions:

1. **The Feature Scale Dependency:** Suppose Feature 1 is house size in square feet ($[800, 4500]$) and Feature 2 is number of bedrooms ($[1, 5]$). If you apply Ridge or Lasso regularization *without* running `StandardScaler()` first, what happens to the weights of the two features? Which feature is disproportionately penalized, and why is normalization mandatory before regularization?
2. **Probability Calibration vs. Overconfidence:** Why does an unregularized logistic regression trained on linearly separable data produce overconfident probability predictions ($p \approx 1$ or $p \approx 0$)? How does adding an L_2 penalty restore calibrated, reliable probabilities?
3. **The Data-to-Parameter Ratio:** In modern machine learning settings where $p \gg n$ (more features than data samples), why is ordinary least squares impossible to compute, and how does regularization (both L_1 and L_2) mathematically resolve the problem?